富卫测试部门 AI 提效方案设计

建议不要做一个“AI 测试平台”，而是建三样能长期留存的资产：一套按开放标准写的测试 Skill 库，一组接入 Jira、日志、造数、CI 的 MCP 连接器，一套用来证明 AI 输出可信的评测数据集。模型和 AI 工具每半年就会换代，这三样资产换模型、换工具后都能继续用，不会被快速替代。

第一批落地的场景按回报排序：失败用例分诊（Badcase 定位）、测试用例设计、测试数据准备。依据是 Meta、Uber 等公司已发表的生产环境数据：AI 生成的测试和修复，在严格的人工审核和自动校验下，被工程师采纳的比例在 50%–73% 之间。这说明 AI 有用，但每一步都必须有自动校验和人工把关。

73%	47.6%	63%	97M+
Meta AI 测试建议采纳率 经过构建、运行、覆盖率三道过滤	工业级 flaky 测试自动修复率 FlakyGuard，ASE 2025	AI 生成用例的 flaky 根因 依赖了不保证的顺序，ICSE 2026	MCP SDK 月下载量 2025年12月，已移交 Linux 基金会

一、设计依据：什么会留下，什么会被替代

1. 三个已成行业标准的底座

已核实

标准	是什么	为什么不容易被淘汰
MCP（模型上下文协议）	让 AI 调用外部系统的统一接口，相当于 AI 的“USB 口”	2025年12月由 Anthropic 捐给 Linux 基金会下的 Agentic AI Foundation，OpenAI、Google、微软、AWS 都支持；ChatGPT、Claude、Copilot、VS Code 都能直接用[1]
Agent Skills（SKILL.md）	一个文件夹 = 一项技能：说明文档 + 脚本 + 参考资料，AI 需要时才加载	开放规范，Claude Code、OpenAI Codex、GitHub Copilot、Cursor、Gemini CLI 等都兼容，写一次可以在多个 AI 工具里用[2]

标准	是什么	为什么不容易被淘汰
W3C WebDriver	Selenium、Appium 底层的浏览器/App 控制协议	国际标准，富卫现有脚本资产都建在它上面

结论：公司自己的业务知识写成 Skill，系统接入做成 MCP，模型选择保持中立。富卫2024年与微软签了四年协议，用 Azure OpenAI 做生成式 AI[3]，所以方案默认跑在公司已批准的 Azure 环境里；以后换模型，Skill 和 MCP 不用重写。

2. 厂商已经在做的，不要自己重造

- Playwright 官方测试 Agent：自带 planner（探索应用、写测试计划）、generator（把计划变成脚本，并实时校验定位和断言）、healer（重跑失败用例、自动修定位）三个 Agent，可接入 VS Code、Claude Code、Codex[4]。注意它生成的是 Playwright Test（TypeScript）脚本，不是 Python。
- Appium 官方 MCP：提供40多个工具，包括会话管理、元素查找（优先 accessibility id）、手势、WebView 上下文切换、截图录屏、权限管理、按自然语言生成测试代码[5]。它生成的是 Java/TestNG 代码，富卫用 Python，要靠自己的 Skill 规定输出风格。
- Atlassian 官方 MCP：可连接 Jira 和 Confluence，支持 OAuth 授权[6]。富卫是否用 Jira 云版，需要先确认。
- 因此会被快速替代的做法有：自己做聊天界面、自己写元素自愈算法、自己封装模型调用。这些不建议投入。

3. 研究证据给出的五条设计约束

科学依据

证据	结论	落到设计上
Meta TestGen-LLM（FSE 2024）：生成的测试 75% 能编译，57% 稳定通过，25% 提升覆盖率；最终 73% 被采纳上线[7]	原始输出有四分之一连编译都过不了，必须先过滤再给人看	每个生成类 Skill 都内置“编译 → 运行 → 多次重跑 → 人工审核”四道关
ICSE SEIP 2026（SAP HANA 等4个数据库）：AI 生成的测试 flaky 比例略高；63% 的根因是依赖了不保证的返回顺序；老用例的毛病会被 AI 照抄[8]	给 AI 看的示例脚本质量决定输出质量	Skill 里只放审核过的“标杆脚本”作示例；新脚本合并前连续跑5次
FlakyGuard（ASE 2025）：只给 AI 最相关的代码上下文，修复了 47.6% 可复现的 flaky 测试，51.8% 修复被开发采纳，根因说明 100% 被认为有用[9]	上下文给太少或太多都会失败	分诊工具先用确定性脚本筛出相关日志和代码，再交给 AI
Westermo 工业案例（PROFES 2026）：用 AI 做夜间回归失败的根因分析，单 Agent 和多 Agent 质量差不多，单 Agent 更快更便宜[10]	多 Agent 编排不一定更好	从单 Agent + 多个 Skill 起步，有数据证明需要再拆
Google DORA 2025（近5000名从业者）：AI 是“放大器”，放大组织原有的长处和短板[11]	流程混乱的团队用了 AI 会更乱	先统一用例模板、缺陷模板、日志 trace ID，再上 AI

二、整体架构

四层结构

层	内容	谁维护
① 使用入口	测试人员在 IDE（VS Code + Copilot 或 Claude Code）里对话使用；CI 里无人值守运行（失败自动分诊）；结果回写 Jira 和 Teams	直接用厂商产品，不自研
② Skill 库	10个测试 Skill（见第三部分），放在 Git 仓库里，走代码评审，有版本号	测试团队 + AI 应用工程师
③ MCP 连接器	外部现成：Playwright MCP、Appium MCP、Atlassian MCP 内部自建：造数 MCP、日志查询 MCP、只读数据库 MCP、CI/测试报告 MCP、费率计算 MCP	AI 应用工程师
④ 知识与评测	业务规则库（产品条款、核保规则、费率表、术语中英对照）；历史缺陷库；每个 Skill 的评测集（标准答案）	测试 + BA + 精算提供数据

关键原则：凡是有标准答案的计算都交给代码，AI 只负责理解、归纳和起草。保费是多少、日期加90天是哪天、两个组合是否覆盖，这些由脚本算；AI 只负责读需求、写说明、给分类。原因是大模型做精确计算会出错，而保险系统的金额错误不可接受。

安全与合规边界

必须先定

- AI 只连接测试环境，生产库禁止接入；数据库 MCP 只给只读账号。
- 日志进入模型前，由脚本先脱敏：证件号、手机号、姓名、银行卡号、病史字段。
- 广州团队处理的是港澳和东南亚市场的系统，跨境数据要遵守内地《个人信息保护法》和各市场法规，具体边界需要与法务、信息安全确认。
- 所有 AI 调用留审计日志：谁、何时、用了哪个 Skill、输入输出摘要。香港保监局已表示会更新《人工智能应用指引》，富卫也是其 AI 计划的七家首批签约保险公司之一[12]，可追溯是基本要求。
- AI 不能自己关缺陷、不能直接合并代码、不能决定能否上线，这三件事只能由人做。

三、Skill 库设计：10 个 Skill

总览

SKILL	解决的问题	输入 → 输出	确定性部分（脚本）	人工把关
req-testability-review 需求可测性评审	验收标准模糊、边界缺失、多市场差异漏写	Jira User Story → 待确认问题清单 + Given/When/Then 草稿	按检查清单逐项比对（年龄口径、币种、时区、市场）	BA 确认后才进开发
testcase-design 用例设计	写用例耗时、组合爆炸、只写正常流程	验收标准 → 用例 CSV（可导入用例管理工具）+ 需求追溯	成对组合用 PICT 或 allpairspy 算，不让 AI 自己排列	测试负责人抽查
premium-oracle 保费期望值	不知道正确保费是多少	产品代码 + 参数 → 期望保费	全部由脚本按精算费率表计算	费率表由精算签发
test-data-factory 测试造数	造一张生效保单要15–30分钟	“要一张过了等待期、香港、港币、吸烟的保单” → 保单号	调用内部造数接口；时间穿越由接口完成	无（只在测试环境）
automation-author 自动化脚本编写	写脚本慢、风格不一致、定位不稳	用例 → pytest + Selenium/Appium 脚本（Page Object）	lint、类型检查、连续跑5次	代码评审后合并
failure-triage 失败分诊（Badcase 定位）	每晚几十个失败，逐个查日志要半天	失败用例 → 分类 + 证据 + 根因假设 + 缺陷草稿	收集截图、Allure 报告、按 trace ID 拉日志、最近提交	测试确认分类后再提缺陷
flaky-fix 不稳定用例修复	时过时不过，没人有空修	flaky 用例 → 根因说明 + 修复补丁	重跑统计 flaky 率	代码评审
defect-writer 缺陷起草与查重	缺陷描述不清、重复提单	分诊结果 → 标准格式缺陷 + 疑似重复列表	Jira 检索相似缺陷	测试提交
regression-selector 回归范围推荐	不知道改动影响哪里	代码变更 → 推荐回归用例集 + 理由	代码调用关系、历史“改了A坏了B”记录	测试负责人确认
test-report 测试报告	写报告和汇报耗时	执行结果 → 中英文测试报告	通过率等数字由脚本统计	测试经理审阅

重点 1：失败分诊 Skill

为什么最先做：它每天都用，风险最低（只读、只出建议），效果最容易量化（每个失败节省多少分钟）。Westermo 和

优先级最高

FlakyGuard 两项研究都证明了这个场景在工业环境可行。

工作流：

- 触发：CI 夜间回归结束，有失败用例时自动启动。
- 取证（脚本完成）：拉取失败截图、页面 HTML 或 App 页面源码、Allure 步骤、接口请求和返回、按 trace ID 查后端日志、对比上次通过时的版本差异与环境变更。
- 先做确定性判断：同一用例近10次通过率低于90%，标记疑似 flaky；同一时间大量用例失败且报连接错误，判为环境问题。剩下的才交给 AI。
- AI 分类：真实缺陷 / 脚本问题（定位失效、等待不足）/ 环境问题 / 测试数据问题 / 需求变更未同步 / flaky，每类必须附证据原文。
- 输出：一页分诊报告推到 Teams，真实缺陷附 Jira 草稿；同一根因的多个失败合并成一条。
- 人工：值班测试确认或修改分类，系统记录 AI 判断对错，作为后续评测数据。

Skill 的文件结构示例（按 Agent Skills 规范）：

```
failure-triage/
├── SKILL.md              # 分诊步骤、分类定义、输出格式
├── references/
│   ├── taxonomy.md       # 6类失败的定义和判定示例
│   ├── fwd-log-format.md # 各系统日志格式、trace ID 规则
│   └── known-issues.md   # 已知环境问题清单（定期更新）
├── scripts/
│   ├── collect_evidence.py   # 拉截图、日志、Allure 数据
│   ├── flaky_stats.py        # 计算历史通过率
│   └── mask_pii.py           # 脱敏
└── assets/
    └── triage_report.md      # 报告模板
```

```
---
name: failure-triage
description: 分析自动化测试失败用例，判断是产品缺陷、脚本问题、环境问题、
  数据问题还是 flaky，并给出证据和缺陷草稿。用于夜间回归失败、
  CI 失败、用户说“帮我看下这个用例为什么挂了”时。
metadata:
  owner: qa-gz
  version: "1.0"
---
## 步骤
1. 运行 scripts/collect_evidence.py --run-id <id> 收集证据
2. 运行 scripts/mask_pii.py 脱敏后再阅读日志
3. 运行 scripts/flaky_stats.py；通过率 < 90% 先标记疑似 flaky
4. 按 references/taxonomy.md 分类，每个结论必须引用证据原文
5. 证据不足时输出“无法判断”，并列出还缺什么，禁止猜测
...
```

重点 2：用例设计 Skill

- 读取 Jira 需求和 Confluence 产品规则，先输出“我理解的业务规则”让测试确认，再生成用例，避免一开始就理解错。
- 按 ISTQB 方法分层生成：等价类、边界值、判定表、状态迁移、异常清单（网络中断、重复提交、超时、会话过期）。
- 参数组合交给 PICT 一类的工具计算，AI 只负责找出参数和取值。
- 期望结果中的金额一律调用 premium-oracle 计算，不让 AI 填数字。
- 输出带需求编号的 CSV，直接导入 TestRail、Xray 或 Zephyr，自动形成需求追溯矩阵。
- references 里放“标杆用例”：10–20条经过评审的高质量用例，作为 AI 的写作范例。

重点 3：自动化脚本 Skill

- 用 Playwright MCP 或 Appium MCP 让 AI 实际打开测试环境、走一遍流程，拿到真实元素，再写脚本，避免凭空编造定位。
- Skill 里规定富卫的写法：pytest、Page Object、定位优先级（data-testid → accessibility id → id → CSS，禁止绝对 XPath）、显式等待、禁止 sleep、每个用例自己造数。
- 合并前自动连续跑5次，任何一次失败就打回；这一步直接对应 ICSE 2026 研究发现的 flaky 风险。
- 网页端技术路线：现有 Selenium Python 脚本继续维护；新建的网页测试项目可以评估 Playwright，因为它官方自带生成和自愈 Agent，但要接受改用 TypeScript 或只用 Playwright Python 的基础能力。这是一个需要团队决定的取舍。

四、怎么证明它真的有用

1. 评测集：每个 Skill 上线前必须过

- 失败分诊：收集过去3个月已有结论的100个失败用例，看 AI 分类准确率；目标先定80%，“真实缺陷被误判为脚本问题”这一类漏判必须低于5%，因为漏掉真 bug 的代价最高。
- 用例设计：选10个历史需求，把 AI 生成的用例与当时人工用例对比，统计漏掉的场景，再看生成的用例能否覆盖当时上线后才发现的缺陷。
- 脚本编写：统计首次运行通过率、5次重跑稳定率、评审修改行数。
- 每次修改 Skill 或换模型，都重新跑一遍评测集。Agent Skills 官方文档也专门有一章讲如何评测 Skill[2]。

2. 业务指标：上线前先测基线

指标	怎么量	对应 SKILL
单个失败用例的分诊时间	从失败到给出结论的分钟数	failure-triage
AI 建议采纳率	被人接受、未修改的比例（Meta 的参照值为73%）	全部
每个需求的用例编写工时	工时记录	testcase-design
造一张可用保单的时间	计时	test-data-factory
自动化覆盖率	已自动化的回归用例数 / 回归用例总数	automation-author
flaky 用例占比	近30天通过率在10%–90%之间的用例占比	flaky-fix
缺陷退回率	开发以“无法复现、描述不清”退回的比例	defect-writer
上线后漏测缺陷数	生产环境发现、本该测出的缺陷	整体质量护栏，不能变差

五、落地节奏

分四期推进

阶段	时长	做什么	验收标准
0 准备	2–3周	与信息安全、法务确认数据边界；统一缺陷模板和日志 trace ID；测量基线指标；准备评测集	合规批复；基线数据
1 试点	6–8周	failure-triage、test-data-factory、defect-writer；选一个产品线、一个团队试点	分诊准确率 ≥ 80%，分诊时间明显下降
2 扩展	2–3个月	req-testability-review、testcase-design、premium-oracle、automation-author、flaky-fix	采纳率、首次运行通过率达标
3 深化	持续	regression-selector、test-report；推广到其他市场团队；Skill 库内部开源共建	回归时长、漏测缺陷数

人员能力：ISTQB 已推出“生成式 AI 测试”专项认证（CT-GenAI v1.1），内容覆盖提示工程、幻觉与隐私风险、LLM 测试方案架构、组织落地路线[13]。可以把它作为团队培训大纲，这也是行业认可的能力标准。

六、需要先确认的事

公开资料无法核实

待确认

- 广州测试团队现在用的用例管理工具、缺陷系统、CI 平台分别是什么（JD 没写）。这决定要接哪些 MCP。
- 公司是否已经批准测试人员使用 GitHub Copilot、Claude Code 或其他 AI 编程工具，以及可用的模型版本。微软合作协议说明 Azure OpenAI 可用，但内部具体开放了什么，需要找 IT 确认。
- 是否已有造数接口、统一 trace ID、日志平台。如果没有，这些属于第0阶段要补的基础建设，它们本身就能提速，与 AI 无关。
- 你说“现在应该有一些 Skill”，我在公开资料里查不到富卫内部已有的 Skill。建议先盘点内部已有的提示词、脚本和文档，把好用的整理进上面的 Skill 结构，不要从零写。

Made with Dia